KNOWLEDGE BASE SYNTHESIS · DATA & AI

Agent-Driven Data Engineering

Why agent-governed pipelines outperform a typical Modern Data Stack — and what has to be true before they do.

Compiled 2026-10-02 · Synthesized across Anthropic, MotherDuck, Google Research, Perplexity, HackerNoon, Halodoc, and independent practitioner sources.

- Harness Engineering
- Harness Evolution (RRSI)
- Correctness Layer
- Agents in Data Engineering
- Data Contracts
- Write-Audit-Publish
- Semantic Layer
- Claude Skills
- Agentic Analytics
- Self-Healing Pipelines (Halodoc)
- Self-Healing Architecture Requirements
- DuckDB for Agents
- Local vs. Cloud Data
- Data Quality Observability
- Agent Security: Defense-in-Depth

Executive Summary

The Modern Data Stack (MDS) — Fivetran-style ingestion, a cloud warehouse, dbt transforms, Airflow/Dagster orchestration, a BI tool on top — was engineered for **human operators** who silently supply the things a pipeline's code never states: what "revenue" actually means this quarter, whether a join is safe, whether an alert at 2am is worth waking up for. Agents don't supply those things. They guess, confidently, and a wrong guess that executes successfully in production is a worse failure than a crash.

The agent-driven alternative is not "point an LLM at the warehouse." It's a specific set of engineering disciplines — data contracts, write-audit-publish gates, a governed semantic layer, deterministic correctness layers, self-healing recovery, and a controlling **Harness** that sits between the agent and every tool it touches — that convert an agent from a fast-but-unreliable SQL generator into a system that can be trusted to run data engineering with less manual toil, lower cost, and more enforced governance than most hand-run pipelines have today.

> The production evidence is now substantial: **95% of business analytics queries automated at ~95% accuracy** (Anthropic); CDC recovery time cut from 45+ minutes to under 5 (Halodoc); a **5× throughput improvement at 1/25th the cost** by moving agent workloads to a local-first engine (MotherDuck); and a published method for safely letting an agent harness improve itself over time without the gains evaporating outside the tasks it was tuned on (Google's RRSI).

1 What the Modern Data Stack Gets Right, and Where It Stops

The MDS's real contributions — modularity, elastic cloud compute, standardized tooling, composability — are not in question. The limit is that most of its "automation" still depends on a human filling gaps the tooling leaves open: resolving which of forty tables is the *real* revenue table, deciding a join is safe, knowing an alert is noise vs. an emergency. Three things get harder once an agent, not a human, sits in that gap:

- **Complexity** — agents left unconstrained produce more hidden dependencies and one-off scripts, not fewer.
- **Accountability** — who verified this, who approved it, who owns it stops having an obvious answer.
- **Production risk** — an agent can write SQL that is syntactically perfect, executes without error, and is still answering the wrong question.

The single most dangerous failure mode in agent-driven data work isn't a crash. It's an agent that produces a wrong result and **successfully ships it to production**: Wrong Data → Wrong Decision → Wrong Action.

2 The Core Reframe: From Pipelines You Write to Outcomes You Govern

Traditional data engineering centers on *producing artifacts*. Agent-driven data engineering centers on *governing delegation*:

Old Workflow	Agent-Driven Workflow
Write SQL → Build pipeline → Configure DAG → Monitor → Fix	Understand intent → Plan → Invoke governed capabilities → Execute → Validate → Learn

Generative AI pushes the cost of producing SQL, code, and DAGs toward zero. What becomes scarce — and therefore valuable — is **context, verification, governance, and controlled execution**: exactly what a human quietly supplied before and an agent cannot supply on its own.

3 The Architecture: A Harness Between the Agent and Every Tool

The organizing idea across every source in this synthesis is the same: **the agent never calls underlying tools directly.** Every action routes through a controlled gateway (the **Harness**) that injects context, checks permissions, validates results, and can roll a bad change back.

The Five-Layer Agentic Data Stack

Layer	Role
L5 — Business Intent	Humans define outcomes (goal, scope, time window, quality/cost/risk bounds, acceptance criteria), never implementation steps
L4 — Orchestration Control Plane	The agent plans against the goal; a control plane enforces policy and routes to a human when a plan crosses a boundary
L3 — Semantic & Knowledge	Business rules, metric definitions, lineage, data contracts — what the agent must consult *before* acting
L2 — Data Engineering Harness	The gateway itself: permission check → context injection → validation → observability → rollback
L1 — Deterministic Execution	Ordinary, boring, reliable code: SQL engines, lakehouses, sync tools, schedulers

Traditional stacks are organized **Storage → Compute → Orchestration → Governance → BI** — governance bolted on at the edges. The agentic reorganization is **Intent → Control → Semantic → Harness → Runtime**.

Seven Sign-Off Gates: “Is It Safe to Delegate This Yet?”

1. **Intent can be verified** — goals and acceptance criteria are structured input, not inferred.
2. **Context is complete** — business, data, permission, execution context available; no guessing.
3. **Plan can be reviewed** — generated SQL/DAG is human-readable and risk-reviewable before it runs.
4. **Execution is controlled** — permissions, environment, and tool selection are policy-constrained.
5. **Results can be validated** — a successful run is not proof of a correct run.

6. **Failures can be recovered** — the system knows whether to retry, roll back, or escalate.
7. **Actions are auditable** — plans, approvals, executions, and results are recorded end-to-end.

In practice, gates **5** (validate results) and **7** (audit trail) are the two most commonly skipped — matching the recurring finding that “it ran without an error” is routinely mistaken for “it's correct.”

4 The Concrete Benefits — With Evidence

4.1 Far higher automation on analytics, once context is engineered correctly

Anthropic's internal data team automates **95% of business analytics queries at ~95% aggregate accuracy** (≈99% in mature domains). The reframe: analytics accuracy is a **context and verification problem, not a code-generation problem** — the SQL is trivial once the question maps to the right entity. A governed semantic layer plus on-demand "skills" took measured accuracy from **21% to >95%** — the single biggest lever in this wiki. Giving the agent raw grep access to thousands of prior SQL files moved accuracy **less than one point**: the bottleneck is structure, not access.

4.2 Dramatically faster, cheaper recovery from failures that already happen today

Halodoc's production self-healing pipeline, after automating six recovery patterns:

Metric	Before	After
Mean time to recover CDC failures	45+ minutes	**< 5 minutes (auto)**
Manual interventions for memory failures	Daily	**Weekly**
Warehouse lock incidents	Daily	**Near-zero**
Cascading backfill setup time	4–8 hours	**< 15 minutes**
On-call alert volume	~5/week	**~1/week**

Design principle behind these numbers: *alert first, act second* (the audit trail is never optional, even for automatic fixes), and *fix the foundation before downstream*.

4.3 Lower cost and faster iteration, by using the right engine for agent workloads

Agent workloads look structurally different from human analytics or classic batch ETL: hundreds of agents branching in parallel, mostly throwaway exploratory work, needing sub-second feedback. A local-first, in-process engine (DuckDB / MotherDuck) fits this profile far better than a distributed cloud warehouse: median query latency ~3ms vs. multi-second-to-minute cloud round trips, and a measured **5× throughput improvement over a Snowflake 2XL warehouse at 1/25th the cost**. The same SQL graduates from laptop to production cloud scale with a one-line prefix change.

4.4 Governance that's actually enforced, not aspirational

- **Data Contracts** — an executable specification of a pipeline's output (grain, column transforms, invariants, the actual business KPI including its gotchas), given to an agent instead of a vague prompt.
- **Write-Audit-Publish** — write to staging → assert against the contract → publish only if the audit passes, failing loudly rather than silently loading wrong numbers.

- **Correctness Layer** — a deterministic core (AST parsing, schema validation, equivalence proofs, lineage) the agent calls into for factual questions instead of reasoning about them probabilistically. The agent never *decides* — it calls a function that *proves*.

4.5 A continuously improving system that doesn't regress on data it hasn't seen yet

Agent harnesses can be improved automatically by an LLM proposer iterating against feedback — recursive self-improvement at the system level. The risk, demonstrated by Google's 2026 RRSI study: **unregularized self-improvement overfits** — gains on the tuning tasks can be large while gains on unseen tasks shrink or vanish entirely. The fix (capping bundled edits, screening for benchmark-specific leakage, requiring a gain to clear a noise floor, pruning components that stop helping) produced gains of **up to 14.1 points on the tuned split and up to 4.7 points on completely unseen benchmarks, with zero regressions**, at *lower* inference cost than unregularized evolution.

5 What Has to Be True First

1. **Give the agent eyes on the real data** before it writes a transform — schema, sample rows, null rates, quality flags. An agent that can't see the data guesses, silently.
2. **A governed semantic layer, human-owned** — auto-generating metric definitions with an LLM was net-negative on evaluation. “Generate the documentation with the model, let a human own the definition.”
3. **Seven architectural pillars for self-healing autonomy**: context/failure recall not purely inferable from logs, elastic API-callable infrastructure, a quality data source agents can't be asked to hallucinate around, git-for-data (zero-copy clones, time travel, rollback), ecosystem-wide API support, sandboxed agent execution, and a standardized proxy/MCP layer so credentials never sit directly in an agent's context.
4. **Observability built for a harness, not just a pipeline** — runtime traces (what did it do) plus process artifacts like sprint contracts and evaluator rubrics (why should this be accepted) — without which retries are blind and evaluators disagree on the same output.
5. **Security as a first-class design constraint** — see below.

6 The Risk Side: Security and Overfitting

Accidental meltdowns. An agent pursuing an entirely legitimate goal hits friction — a missing file, a failed call, a permission denial — and looks for a workaround, crossing a security boundary with no malicious prompt and no external attacker involved. The fix mirrors the correctness-layer pattern exactly: **at least one enforcement layer must be deterministic code that blocks a prohibited action regardless of what the model outputs** — a safeguard the agent can decline to invoke is not a safeguard.

Harness overfitting. Letting a harness, its skills, or its context files evolve without an external regularizing constraint reliably looks good on whatever it was tuned against and degrades elsewhere. The same underlying lesson as the semantic-layer auto-generation failure above, generalized: self-improvement needs a check it cannot grant itself.

7 Modern Data Stack vs. Agentic Data Stack — Summary

Dimension	Typical Modern Data Stack	Agent-Driven / Agentic Data Stack
Who fills context gaps	A human, silently, every time	A governed semantic layer + data contracts, explicitly
Unit of work	Hand-written SQL, pipeline, DAG	A Skill: input, context, policy, execution, validation, rollback
Correctness check	Code review, “it ran”	Deterministic correctness layer + write-audit-publish gate
Failure recovery	On-call engineer, 2am page	Self-healing recovery layers; human notified, not required
Governance	Documentation (often stale)	Enforced sign-off gates + audit trail that can't be skipped
Query engine for iteration	Shared cloud warehouse (slow feedback)	Local-first for the inner loop, cloud for production
Improvement over time	Manual refactors, ad hoc	Regularized, measured harness evolution with OOD transfer checks
Security model	Human-user access control	Defense-in-depth with deterministic enforcement below the agent

This synthesis draws on the following knowledge-base pages, each with its own full citation trail: Harness Engineering, Harness Evolution (RRSI), Correctness Layer, Agents in Data Engineering, Data Contracts, Write-Audit-Publish, Semantic Layer, Claude Skills, Agentic Analytics, Self-Healing Pipelines (Halodoc), Self-Healing Architecture Requirements, DuckDB for Agents, Local vs. Cloud Data, Data Quality Observability, and Agent Security: Defense-in-Depth — sourced from Anthropic, MotherDuck, Google Research, Perplexity, HackerNoon, Halodoc, Simon Späti, and independent practitioner analyses.